

Individual Game Submission

CSC 481/581 - Game Engine Foundations - Fall 2026

Group 4: Yeva Mheryan

Milestone 1 - Build Game Engine Foundations

Module 3 - Game Play Foundations

Individual Game Name: “Hello Kitty Adventure”

GitHub Game Repository:

<https://github.com/CodeRockerr/CSC481-581-Group4/tree/main/Games/ymherya>

(A) Introduction

For Milestone 1, my primary objective was to use the game engine from our team and create my own game with the necessary features. For that, I developed a game based on the popular character Hello Kitty, called Hello Kitty Adventure. In it, the player controls Hello Kitty, and Kuromi moves throughout the level as the enemy. Some other features of this game include a background, player movement, jumps, gravity, animations, collisions, and window resizing. The idea behind my game was simplicity of the gameplay along with demonstration of how our engine works. As opposed to implementing all of the systems in one place, the engine divides entities, input, physics, rendering, collisions, and scaling into different systems and my game uses these systems to produce actual gameplay.

(B) Shared Engine Architecture

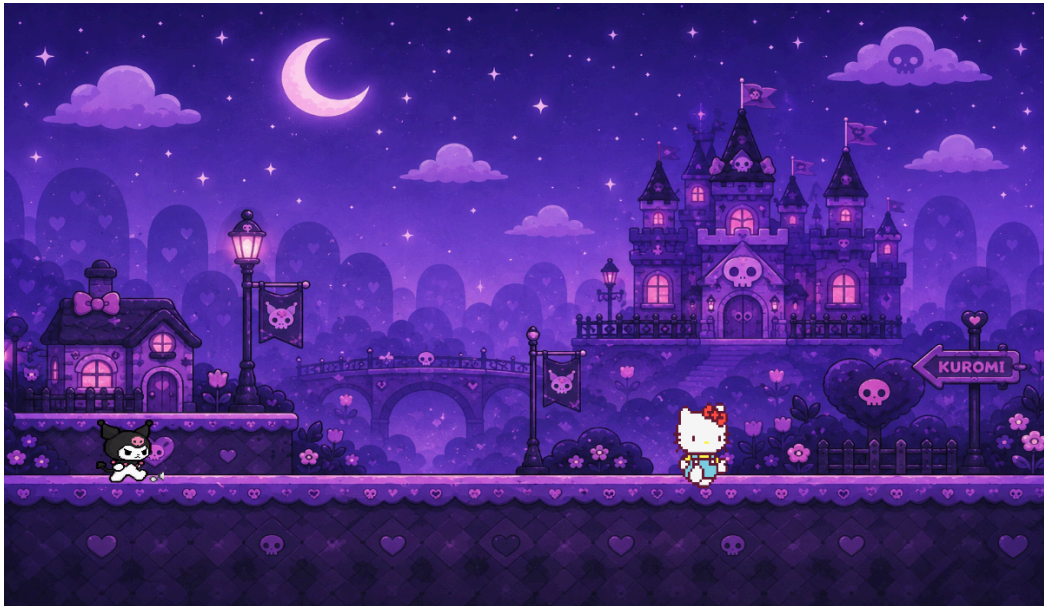
The engine was designed in a way that the same basic systems could be used by all games of individual members. The Engine class manages the game loop, whereas the other classes take care of different elements of the game. For instance, the Entity class contains data on the object's location, size, velocity, textures, and animations. The EntityManager takes care of creating and updating those entities and also draws them. I appreciated the separation of the systems because of the reason that game-specific code would not have to do everything on its own. My game does

not require me to develop my own rendering system or SDL window because there are already systems created by our team that can be used by my game. All that I would have to do is figure out how Hello Kitty and Kuromi should act in the game. Another feature of the engine is that it uses delta time while updating movement. It means that movement is calculated in accordance with the time elapsed instead of movement by a certain number each frame.

(C) Task 1 - Engine Initialization and Main Loop

(a) Implementation

Hello Kitty Adventure is the name that I use when I generate the engine. This engine opens the window and the renderer and begins the game loop. The behaviors that are specific to the game are defined in the callback that is given to `engine.run()`. In the loop, the engine will handle window events, update the input, process the physics, update the entities and render the game. I decided on this structure since it will put the game loop into the engine and not require every game to have one. My `main.cpp` simply instantiates my game objects and tells them what to do.



(D) Task 2 - Required Game Entities

I used three main entities in my game.

(a) Static Entity - Background

Background is the static entity for me. I have loaded the background image and attached it to the entity through the engine's texture system. The background remains at one place and does not react to any user input as well as physics. I have created the background so that it covers the entire gaming area as the game environment.



(b) Controllable Entity - Hello Kitty

Hello Kitty is the character which is controlled by the player. An 8-frame sprite sheet has been used for Hello Kitty and linked to the EntityManager. The player can control the movement of Hello Kitty to the left and right through either A/D key or Arrow Keys. Jumping has been enabled in order to make Hello Kitty move vertically. When she is moving, the sprite of Hello Kitty changes with animation frames. When she is at rest, the first frame is used and another frame is used when she is flying.



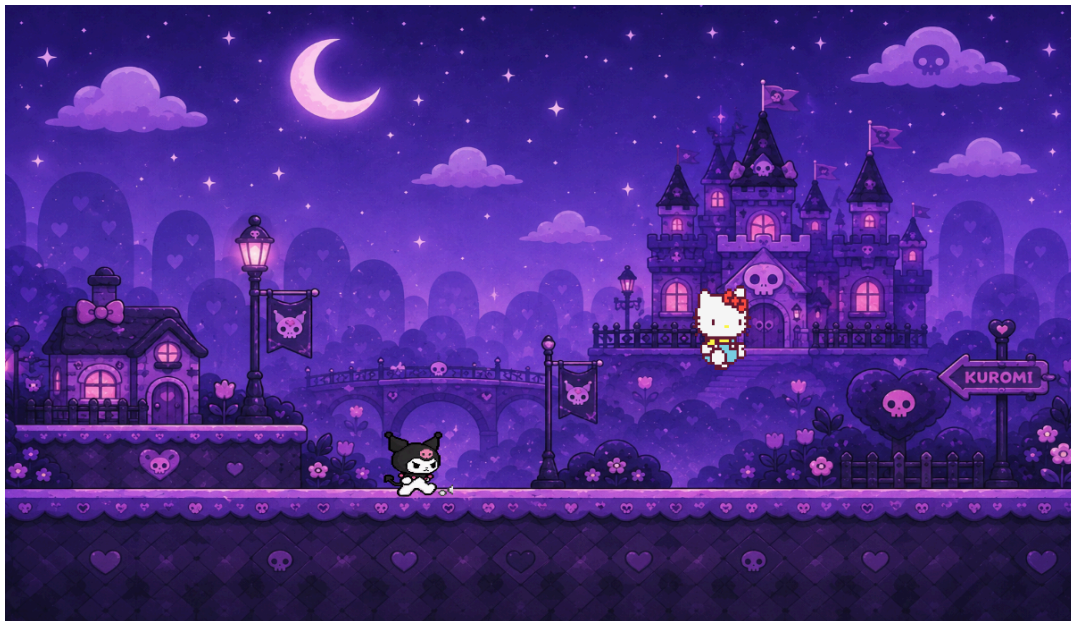
(c) Automatically Moving Entity - Kuromi

The automatically moving character in my game is Kuromi. Similarly, Kuromi also uses an 8-frame sprite sheet. However, unlike the main character, Kuromi doesn't require player input to move. Rather, she constantly moves from one point to another. Once she hits either of those points, she changes her course of movement and moves back in the opposite direction. Her movement is done using delta time, which ensures that she always moves at a constant pace.



(E) Task 3 - Physics and Gravity

The physics system within our engine allows me to apply the force of gravity on the entities individually. It works in my favor as far as Hello Kitty is concerned since she will need to be able to jump and fall. As soon as Hello Kitty jumps, I give her a vertical velocity upward. Then the force of gravity works on her through the physics system. This means that she slows down on her upward motion and falls back to the ground eventually. The force of gravity was applied to Hello Kitty alone because Kuromi follows a certain trajectory horizontally and the background has no need to move at all.



(F) Task 4 - Keyboard Input and Player Controls

I used the shared Input system for my player controls. Instead of checking keyboard events directly in my game, I use the keyboard-state functions provided by the engine.

The controls for my game are:

Input	Action
A/Left Arrow	Move left
D/Right Arrow	Move right
W/Up Arrow/Space	Jump

I use `isKeyPressed()` for movement because the player should continue moving while the key is held down. I use `isKeyJustPressed()` for jumping because I only want a jump to happen when the key is initially pressed. I also added boundaries on both sides of the game so that Hello Kitty cannot move completely off the screen.

(G) Task 5 - Collision Detection and Game Responses

The collision detection mechanism detects if there are two overlapping objects. The decision of what action will take place upon collision detection is left to the game logic. In my case, the primary collision among enemies is the collision between Hello Kitty and Kuromi. Upon collision between the two entities, Hello Kitty is brought back to her initial position, and her vertical velocity is set to zero. In this way, I make sure that the collision actually makes some changes in my game, not only detects the touching of the two objects. Ground collision is applied to Hello Kitty as well. If she is falling down and touches the ground, her position is corrected so that she is on the right level, and her vertical velocity becomes equal to zero. I have made the decision to keep collision detection and collision response separate because various games might require different actions upon collision detection.

(H) Task 6 - Runtime Scaling

I have also used the scaling feature that we had implemented in the engine. In my game, I have a logical game space with a 1280x800 resolution. Instead of making use of the current window size for object creation, I have made use of the game resolution itself for creating objects. Thus, the `EntityManager` scales the objects appropriately while drawing them. This ensures that if the game window is bigger or smaller, the objects can resize themselves with the window, keeping the same layout intact. This was done by me so that the gameplay positions remain constant even as the window size changes.

(I) Design Limitations and Future Improvements

Because this is the first milestone, the game engine is relatively simple. For instance, the collision system employs very simple rectangular hit boxes that sometimes don't line up perfectly with the character sprite images. Similarly, the animation system is also very simple and it relies on a timer system for switching sprites. On the other hand, the physics system currently only deals with gravity and jumping, although it could be made more complex by implementing features such as friction. The scaling system should also be enhanced to ensure that the aspect ratio remains constant whenever the window size changes. For the future, I would like to include additional elements such as enemies, collectibles, and score.

(J) Conclusion

My individual game makes use of the shared engine to generate a basic playable Hello Kitty game. The Hello Kitty character serves as the player, the Kuromi character serves as the moving enemy, and the background serves as the static environment. The features included in the game are movement, jumping, gravity, animation, collision detection, and scaling. Through building this game, I got to know how the various engine systems could operate together without having to put all the functions within the game code. I got to use the engine systems that we had built and also added my own behavior to it.